The Average Monthly Recurring Revenue Of Software As A Service Companies Remains Difficult to Predict

Ye Htut Maung

April 2025

1 Introduction

Over the last two decade, technology has been developing at a fast pace. It changes the way we used to work in our daily life. Especially after the COVID pandemic in 2020 hits worldwide, We rely on computer software more than before. As an example, Zoom, which is an online proprietary videotelephony software program, becomes the most used software during pandemic as we need to quarantine at our home and all education program and most of the work moved from physical to online present. When you use Zoom software before, you notice that there are free plan and premium plan, which has more features than free plan but you need to may every mouth to be able to use it which we call it subscription. You may notice that most of the software that you are using is subscription based and not one time purchase. That means you do not own the software and you are just using as you need. So, Why does subscription based product become popular? We are going to look at in both perceptive, consumer and producer.

As a consumer, the first thing is that using subscription is cheaper than one time purchase or buying a product. As an example, for Adobe software in 2016, the one time purchase of Photoshop is $699 and the subscription fee to use Photoshop is only $9.99 per month. Not like one time purchase, you will also get constant update and other features like customer service for the software. So, you don't need to buy software again to get

an update version. As producers point of view, they have consistent and predictable revenue, long term profit, costumer loyalty, easier to maintenance and reduced piracy. That is why you will see many software that offers subscription. We called subscription based companies as Software as Service or SaaS companies. How do we measure the success or the revenue of SaaS?

One of the most important financial metrics is Monthly Recurring Revenue(MRR). MRR represents the total predictable revenue a company expects to earn from its active subscription every month. Since most SaaS company subscription is monthly basis, MRR can provide clear and consistent way to track how much the company generate revenue regularly. For example, if we oversimplified the calculation of MRR and a company has 1,000 customer and the average monthly plan is $10, then its average MRR is 10,000. This metric is valuable because it helps business forecast revenue, make informed budgeting decision, and track growth over time. Unlike one time purchase, where the revenue is unpredictable on sale, MRR reflects the stability and long-term sustainability of a company's financial performance. As a result, investors and analysts closely monitor MRR to assess how well a SaaS company is performing.

Thus, there is a clear connection between, subscription pricing, customer behavior and the financial success of a SaaS business. A natural question to ask and the focus of this paper is: can we reliably predict the average monthly recurring revenue a SaaS company earns based on observable features such as user count, churn rate or marketing spend? Since MRR is measured month by month, modeling it directly would require repeated measures methods, which is more complex to do. To simplify, we are going to predict average MMR over a year, which is a single, stable numeric value per company. If this quantity can be predicted accurately, it could help companies optimize their strategies for growth, guide investor decisions, and better allocate resources. We will explore this question herein.

In Section 2, we will define the core phenomenon and discuss the motivation behind modeling it. Section 3 will introduce the mathematical framework, including the response variable and chosen features. Sections 4 through 7 will describe the supervised learning

process and its necessary components. Section 8 will discuss the model selection problem while Section 9 will outline some potential use of real-world applications of the model. Finally, Section 10 will provide concluding thoughts on the accuracy and usefulness of modeling average MRR in SaaS businesses.

2 Phenomena and Models

A phenomena is the real world process or behavior we are trying to understand or predict. We can also say that it is reality, absolute truth or system. For instance, the speed of a car, streets in the city, sleep quality, the height of a person and so on. In our case, the phenomenon is the average monthly recurring revenue over the past year of a SaaS company: given the active user and percentage of user on premium plan, what is the average MRR of a company? So, given our circumstance, we will know how successful a SaaS company can be. To understand both phenomenon and the prediction to the future, scientists construct a model for it which is the approximation to the reality. A model is not a perfect scale to the reality. It is built by collecting metric from the real world process. A model globe is the approximation of the earth. A map is the approximation to the roads and streets. A Newton law of $F = ma$ is also not perfect but if you know the approximate force and the approximate mass, you can predict its force. So, you may come up with the question of "why do we use models even though they are not perfect?" The answer is it is useful to use in our life. Maps are not accurate but it is useful to navigate around the places.

While there are many models, mathematical models are the models that you can measure by using metrics. Regarding to our phenomenon, which is monthly recurring revenue, it can be present as numerical values and all the features that can approximate or predict average MRR can also be present as numerical values and we will see those factors in detail in section 3. In our case, a model is to predict average MRR based on the factors that can affect the changes in average MRR. In general, the response measurement or output metric of a phenomena is denoted mathematically as y, such that $y \in \mathcal{Y}$ the set

of all possible output measurements. The average MRR over the past year of a company in U.S. dollar is our response y and since it is income, our response space is in the set of real number which is greater than or equal to 0.

2.1 Purpose and Use

In our context, the model could be used by startup founders, venture capital analysts, or SaaS-focused investors to estimate a company's future AMRR or assess how well a SaaS company is performing based on the feature metrics such as churn, CAC, and growth rate. This could support investment decisions, human resource management, budget allocation to different department of a company or internal forecasting.

When people use our model, they need to be aware of the threshold for "usefulness" for our context. A threshold for usefulness of our model is achieving a prediction error that is substantially lower than that of the null model. The null model is a naive predictor without any feature and only look at the $\boldsymbol{y}$ responses. In our case, the null model is the average AMRR for all companies. As a concrete benchmark, we need to measure with RMSE which is root mean square error and we will talk more about in the section of error metrics. However, a normal person cannot easily understand the term of error. So, in our case, RMSE of $10,000 ,that when the investor uses our model, the real value and our predicted value can be differ by $10,000, is a good threshold as when you gain or lose about $10,000 in financial investment is ignorable.

3 Average Monthly Recurring Revenue of a year as a Function

It will become useful at this time to employ a common metaphor from mathematics, that of the function machine. Think of our model as a black box: there will be an input data that put into the black box and process then you will get prediction output response. Before we define our input that will be use in modeling, let's take a look at our previous definition of calculation monthly recurring revenue. We talked about that if we multiply

the average number of customers and average monthly plan, we can get MRR. Then, as we are going to predict average monthly recurring revenue, we will add all 12 months of MRR and divide by 12. However, this is not perfect to calculate average MRR(AMRR) as it is oversimplified to all the features that account into producing monthly revenue. There are casual drivers that really effect AMRR. So, what does mean casual driver? A casual driver is the true underlying cause of the response y; part of the real-world system, even if unmeasurable. Mathematically, we present them as z's or $(z_1, z_2, \ldots, z_q)$. To illustrate this point consider some reasonable casual drivers for our phenomena of interest:

z_1 = Customer satisfaction

z_2 = Product quality

z_3 = Marketing effectiveness

z_4 = Market demand

z_5 = Economic Condition

z_6 = Customer Support Quality

z_7 = Trust to the company

It is easy to see how we could measure these different casual drivers. However, they are fundamentally unknown or impossible to measure to us as there could be more true drivers to the phenomena. If you take a look at z_5, economic condition, you can never predict it or it is difficult to measure the economic condition. Similarly, z_7, that trust can never be able to measure with metric since it is the feeling of customers.

What we can do with these casual drivers is that we can pass them into an exact functional relation t which will give the phenomena. We can present mathematically as

$$y = t(z_1, z_2, \ldots, z_q)$$

The question is "do we know what an exact functional relation is?" and the answer is no. So, the best that we can do is come up with another machine or function f which can approximate t or the best functional relationship we can get from the input. Furthermore, we can look for approximations of our casual drivers that are measurable. To express

mathematically, we are going to select x's or $(x_1, x_2, \ldots, x_n)$ to act as proxies, which we can call as features, attributes, characteristics, independent variable, explanatory variable or predictors, for our $(z_1, z_2, \ldots, z_q)$ and denote a new rule f which accounts for the fact that t is beyond our understanding. Using these new ideas we obtain:

$$y = t(z_1, z_2, \ldots, z_q) = f(x_1, x_2, \ldots, x_p) + \delta$$

In the above equation, We introduce a new mathematical letter, δ. We call it "error due to ignorance" or **"ignorance error"**. The reason is that we cannot determine t from f directly and there is an error of ignoring real features that can predict the true z's. To reduce ignorance error, we can add more features that can approximate to z's. Our features, x's need to be independent variables which mean that they don't depend on each other. For example, if x_1 is the measurement of table in inches and x_2 is the measurement of table in centimeters, x_1 and x_2 are dependent on each other and in our features, we cannot have like that. Mathematically, we call it as full rank. Before explaining about full rank, let's take a look at our features that could act as proxies for some of the underlying casual drivers of our phenomena~~:—~~. In our setup, the input features $x_1, x_2, \ldots, x_5$ are called independent variables, and the target we aim to predict, the average monthly recurring revenue, is the dependent variable.

x_1 = Customer Churn Rate (Measure how many customers leave in percent)

x_2 = Number of Active Users

x_3 = Customer Acquisition Cost (How much it costs to get a new customer)

x_4 = Percent of user on premium plans

x_5 = Revenue Growth Rate (Measure in percent)

~~The features selected for this model are all numeric and can be measured using data commonly available in financial reports or SaaS~~ All of these feature measurements are both practical and realistically obtainable. They are standard metrics tracked by most SaaS companies and can usually be extracted from internal dashboards, financial statements, or customer analytics tools. ~~Each measurement are practical and can be make accurately as many SaaS companies implement the collection of data in their software. It~~

~~will be~~ In many cases, these metrics are already collected automatically as part of regular operations. While some variation in accuracy may exist depending on how companies report their data, overall, each of the five features can be measured consistently and with high reliability. It is usually just a matter of time or money to ~~get~~ obtain these data. In the following, we will explore ~~about~~ how to calculate each ~~features~~feature.

- x_1 = **Customer Churn Rate**, measured as the percentage of customers who canceled their subscriptions over the past year. It is computed by:

$$x_1 = \frac{\text{Number of Customers Lost in a Year}}{\text{Number of Customers at Start of Year}} \times 100$$

- x_2 = **Number of Active Users**, defined as the total number of paying customers a company has at the end of the year.

- x_3 = **Customer Acquisition Cost (CAC)**, calculated as the average cost to acquire a new paying customer:

$$x_3 = \frac{\text{Total Sales and Marketing Cost}}{\text{Number of New Customers Acquired}}$$

- x_4 = **Percent of Users on Premium Plans**, which is the proportion of customers on paid plans:

$$x_4 = \frac{\text{Number of Premium Plan Users}}{\text{Total Number of Users}} \times 100$$

- x_5 = **Revenue Growth Rate**, measured as the percentage increase in total revenue compared to the previous year:

$$x_5 = \frac{\text{Revenue}_{\text{Current Year}} - \text{Revenue}_{\text{Previous Year}}}{\text{Revenue}_{\text{Previous Year}}} \times 100$$

All five features are expressed as real, non-negative values and are chosen to reflect meaningful aspects of SaaS company performance. Since all features are not categorical, we do not need to transform our features into numerical and dummified. All of the five

features are independent on each other and it must be full rank, which means that we cannot get one feature by linear combination of another feature. If we can get one feature from another feature, those features are dependent on each other and we can also call it collinear and we cannot have multicollinearity in our features.

4 Learning From Data

After we know about our features $(x_1, x_2, \ldots, x_5)$, how do we best comnbine these features using machine, rule or function to produce the average monthly recurring revenue. We cannot solve with analytic approach as we don't know what kind of calculation to use for those features. So, we need to use empirical approach and in this case, we can use supervised learning using historical data. To present historical data in mathematically, we represent as $\mathbb{D}$. $\mathbb{D}$ is the matrix that include all the features and the response as columns and all the rows as data point. It has the dimension of $n \times p$ where n is the number of rows or tuple or our data points and p is the number of columns or features. In our case of SaaS companies, we can get the necessary data from financial institution with necessary cost to assess their data. Each row is a company data for $(x_1, x_2, \ldots, x_5)$ and y.

Machine learning is a field of computer science that focuses on developing algorithms that allow computers to learn patterns from data and make decisions or predictions without being explicitly programmed. Supervised learning is a machine learning technique or a process of finding a function that uses labeled datasets (historical data) to train algorithm models or function to identify the underlying patterns and relationships between input features and outputs. To learn this function, we use an algorithm that analyzes historical data, often known as training data and searches through a predefined set of possible functions, which is called the hypothesis set, to find the one that best fits the data according to some evaluation criteria. Mathematically, this can be expressed as:

$$g = \mathcal{A}(\mathbb{D}, \mathcal{H})$$

where $\mathcal{A}$ is the algorithm, $\mathbb{D}$ is the training dataset, $\mathcal{H}$ is the set of all candidate functions (the hypothesis space), and g is the function that the algorithm selects as the best approximation of the true relationship between inputs and output.

As we talked that supervised learning is a machine learning technique, machine learning in general is a computer takes huge amount of data and using that data, it will find a pattern and learn about the data without being programmed what to do or analytic approach. It is an important learning because humans take a lot of time or sometimes impossible to find a pattern in the million or billion of data points whereas a computer can perform a lot faster than a human can do.

Before we move on, we need to be aware of stationarity. Stationarity is a state that the real cause t and casual drivers $t(z_1, z_2, \ldots, z_q)$ and the function$f$ and its features $(x_1, x_2, \ldots, x_p)$ do not change over the period. When we take a look about our phenomenon (AMRR) and its casual drivers and the function$f$ and its features $(x_1, x_2, \ldots, x_5)$, it can change over the time as there must be other non-stationary factors that effects a revenue of a company such as the politics, economics or natural disaster. If our t and f always change, our g will drift. So, ~~we~~ our model is not stationary as we do not know what new features will contribute to predict our phenomenon in the future. However, for the sake of simplicity, we are going to assume stationarity throughout our modeling.

We now examine the hypothesis space $\mathcal{H}$ in detail. $\mathcal{H}$ is the set of all candidate functions, h which we consider to approximate f, the optimal function. Within $\mathcal{H}$, there is an optimal function that approximate to f and we call it h^*. Since it is an approximation to f, there is an error and we call it, **misspecificatoin error** $(h^* - f)$. Now, we can write in mathematical equation.

$$y = t(z_1, z_2, \ldots, z_q) = f(x_1, x_2, \ldots, x_p) + \delta = h^*(x_1, x_2, \ldots, x_p) + \epsilon$$

However, it is impossible to find the best optimal function h^*. So, we need to find a way to approximate function which we will call it g that is close to h^* and supervised learning comes in. As we talk before, to find g, we use historical data $\mathbb{D}$ and the candidate set $\mathcal{H}$ and put them into an algorithm. The algorithm will give you the best possible

approximate function g. Since we cannot find h^* and, g is just an approximation to it, we have **estimation error** $(g - h^*)$. Then, we will define e as a residual error which include all three errors (ignorance error, misspecification error, estimation error). We can update our mathematical equation as following

$$y = t(z_1, z_2, \ldots, z_q) = f(x_1, x_2, \ldots, x_p) + \delta = h^*(x_1, x_2, \ldots, x_p) + \epsilon = g(x_1, x_2, \ldots, x_p) + e$$

4.1 The Set of Candidate Functions $\mathcal{H}$ Our Model

Previously, we define the set of Candidate Functions $\mathcal{H}$ in general and we are going to examine how we are going to define our candidate functions set for average monthly recurring revenue. What the set $\mathcal{H}$ looks like will depend on a hypothesized relationship between our features and output metric. In SaaS companies, the number of active users is mostly positive correlated to average monthly recurring revenue. When you take a look into some SaaS companies like zoom, dropbox or saleforce, even though they have free version, if there is more users, more of the users buy their premium services. Since they are correlated, they have a linear relationship between them and they are the simplest to start with. You can also imagine a line tracing through a graph of number of active user and its average monthly recurring revenue. We could represent such a relationship mathematically as follows:

$$h(x_2) = b_0 + b_2 x_2$$

If we restricted ourselves to this feature x_2 only, our candidate set would be:

$$\mathcal{H} = \{w_0 + w_2 x_2 : w_0, w_2\} \in \mathbb{R}$$

all possible lines representing the linear relationship between the number of active user and average MRR.

However, when you recall, we selected more than one feature for use in our model and

naturally we would want to include all of them in the same way. We could generalize our $\mathcal{H}$ to be the best of hyperplane, which is a fancy math term for a plan in higher dimensions:

$$\mathcal{H} = \{\mathbf{b} \cdot \mathbf{x} : w_0, w_1, \ldots, w_5 \in \mathbb{R}\} = \{w_0 + w_1 x_1 + \cdots + w_5 x_5 : w_0, w_1, \ldots, w_5 \in \mathbb{R}\}$$

w's are the weight of each feature meaning that how strong this feature relationship with the average MRR is. We can represent perfect weight with each feature as following where β represents the perfect weight:

$$h^* = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \beta_5 x_5$$

However, it is impossible to find the perfect weight for each feature. That's why supervised learning comes in. Supervised learning will search through $\mathcal{H}$ and return a best guess at the value of each $\mathbf{b}$ and return a $\hat{\mathbf{b}}$ which tell us approximately how much changes to each feature measurement contributes generally to our response metric of interest. To run this "search" on our $\mathcal{H}$, we need training data to analyze.

4.2 Training data $\mathbb{D}$

In the beginning of the Section 4, we talked briefly about historical data $\mathbb{D}$ and now we are going to set up $\mathbb{D}$ for the training. Each row represents a single SaaS company and contains values for all five features x_1 through x_5 as well as the response y. For example, one row of data look like the following:

x_1	x_2	x_3	x_4	x_5	y
15.2	12,000	120.50	35.0	22.8	108,000

These measurements represent a company with 15.2% churn, 12,000 active users, a CAC of $120.50, 35% of users on premium plans, 22.8% revenue growth, and AMRR of $108,000. Historical data $\mathbb{D}$ can represent as a matrix with the size of $n \times p$ including

features and a response where n is the number of rows or the number of companies that we have and p is the number of columns.

To start the computation, we need to split the features columns which is called X from x_1 to x_5 and a response columns $\boldsymbol{y}$. Then, we add an intercept column with 1's in the first column of X. That allows the model to learn a bias term and shift the regression surface vertically to better fit the data. Let's take a look what will be looked like as a matrix:

$$X = \begin{bmatrix} 1 & 15.2 & 12000 & 120.50 & 35.0 & 22.8 \\ 1 & 12.5 & 8000 & 105.75 & 40.0 & 19.4 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & x_{n3} & x_{n4} & x_{n5} \end{bmatrix} \quad \text{and} \quad y = \begin{bmatrix} 108000 \\ 74500 \\ \vdots \\ y_n \end{bmatrix}$$

As we mentioned earlier, these data could be collected from financial report, and some commercial sources. A realistic sample since n could range from 5,000 to 10,000 companies. While some metric like number of users and growth rate may be publicly available, others such as churn and CAC may require internal access or subscriptions. That could make the process feasible but potentially expensive and time consuming.

The compilation of historical values stored in X and the corresponding $\boldsymbol{y}$ we have constructed above are known as **training data**, denoted $D = \langle X, \boldsymbol{y} \rangle$. With our $\mathbb{D}$ and $\mathcal{H}$ in hand we are almost ready to model our phenomena. One final ingredient is necessary to carry out the supervised learning process, we need an algorithm $\mathcal{A}$

4.3 Algorithm $\mathcal{A}$

The final component needed for supervised learning is the algorithm, denoted $\mathcal{A}$. A general definition of algorithm is a well defined sequence of finite steps for solving a particular problem. In our case, an algorithm is a procedure that takes in the training data $\mathbb{D} = \langle X, \boldsymbol{y} \rangle$ and a hypothesis set $\mathcal{H}$, and search a best possible function $g \in \mathcal{H}$ that

best fits the data. That is, we write

$$g = \mathcal{A}(\mathbb{D}, \mathcal{H}),$$

which represents the function the algorithm learns from data. The choice of algorithm depends on the context of the problem and the structure of the hypothesis set.

There are many choices of $\mathcal{A}$, such as minimization of mean absolute deviation, tree model, random forest or other error-based procedures. In our modeling, we choose the Ordinary Least Squares (OLS) algorithm. OLS is one of the most widely used algorithms because of its simplicity and interpretability. It is appropriate for our context because our hypothesis set $\mathcal{H}$ is the set of linear functions, and OLS finds the best linear function that minimizes the sum of squared errors between the observed values and predicted values.

Mathematically, the sum of squared errors (SSE) is given by:

$$\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The predicted values $\hat{\boldsymbol{y}}$ are equal to $X\hat{\boldsymbol{b}}$, where X is the design matrix and $\hat{\boldsymbol{b}}$ is the coefficient vector that minimizes the SSE. The OLS algorithm finds the optimal coefficients by solving:

$$\hat{\boldsymbol{b}} = \arg\min_{b} \ \|\boldsymbol{y} - X\boldsymbol{b}\|^2$$

Once $\hat{\boldsymbol{b}}$ is found, the final output function g is given by:

$$\mathcal{A}(\mathbb{D}, \mathcal{H}) = g \Rightarrow g(\mathbf{X}) = \hat{\mathbf{y}} = \mathbf{X}\hat{\mathbf{b}}$$

This function g is the result of the modeling process and serves as our prediction function. It can be used to estimate the AMRR of new companies given their feature measurements.

A **prediction using a model** refers to the process of applying the learned function g to new, unseen feature values in order to estimate the corresponding response. In our

case, for any new SaaS company represented by a feature vector $x^* \in \mathbb{R}^p$, we compute the predicted AMRR as:

$$\hat{y} = g(x^*) = x^{*\top}\hat{\boldsymbol{b}}.$$

Before we move to error metrics, it is important to analyze the sources of prediction error that may arise in our context. As we explain that the sources are ignorance error, misspecification error and estimation error. Among them, ignorance error will be the largest. Although we select five important features, AMRR can be influenced by many other factors that we do not capture in our model. We can reduce it by researching and adding more features that contribute AMRR. We can expect that misspecification error will moderate to large as we use linear relationship in our candidate set as the true relationship between AMRR and our features could not be perfectly linear and be non linear relationship. However, we will reduce this error in model selection section as we will make the candidate set more expensive and complex. We can expect estimation error to be large or small depends on how many data points we can collect.

5 Error Metrics

After generating the model g, we need to measure how good our model is or the performance of our model since our predictive models cannot give the exact response y. So, there is an error between the exact value and our predicted value. We can call that error as prediction error. A **prediction error metrics** is a way of quantifying how accurate your models predictions are.

The most basic way to assess the model is by measuring how far off its predictions $\hat{y}_i$ are from the true values y_i. This difference is called the **residual** and is defined for each observation i as:

$$e_i = y_i - \hat{y}_i$$

The residual tells us how much error the model made on a single prediction. We can collect all the residuals into a residual vector:

$$\boldsymbol{e} = \boldsymbol{y} - \hat{\boldsymbol{y}}$$

If the residuals are small in magnitude across all observations, the model is said to fit the data well. So, How can we represent as a single error metrics for all the observations? This is where SSE, MSE and RMSE comes in.

SSE is called sum squared error and it squares each residual and add all of them, which is defined as:

$$\text{SSE} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

However, SSE will grows with the sample size as it adds the squared residuals over all n companies and it is difficult to interpret. So, we can take the average of our sum error which we call mean squared error or MSE and is defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{\text{SSE}}{n}$$

Now, we get the average error of each company but since we square the error the unit will also be squared which we now have dollar square and it does not make sense. Finally, to return the error to the same unit (dollar) as the response variable, we take the square root of MSE, resulting the root mean squared error or RMSE and is defined as:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

In our context, RMSE measures the typical dollar amount by which our prediction of a company's AMRR differs from the true AMRR.

Another important way to evaluate a model's performance is to measure how much of the total variation in the response variable the model is able to explain. Let's take a look of three quantities which are SST, SSR and SSE. SSE is the same as we mention before

and it can also interpret as the estimate of error variance that remains unexplained by the model. SSR is the sum square regression, which measure the estimate of error variance that explained by the model that can represent as $\text{SSR} = \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2$. SST is the sum squares total, which measure the total variation of the observed responses around the mean that can represent as $\text{SST} = \sum_{i=1}^{n}(y_i - \bar{y})^2$. These 3 quantites satisfy the identity of $SST = SSR + SSE$.

Using this decomposition, we can define R^2 which measures the estimate of the proportion of variance explained by the model:

$$R^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}}$$

The value of R^2 is always between 0 and 1. A value of 1 indicates a perfect fit, where SSE is 0 (RMSE is also 0), while a value of 0 means the model is bad and does not do better than predicting the mean of the observed response, where $SSE = SST$ and basically it is null model (RMSE is large). So, we can say the the higher the R^2 and the smaller the RMSE, the better the model is and vice visa.

So, what error metric should we use in our context. We have 2 best error metrics which are RMSE and R^2. R^2 seems easy to read as it can represent as the percentage of how good it is. However, the downside of R^2 is that even R^2 is high as near 0.9, $RMSE$ can be small by the formula but $RMSE$ can be \$100,000 which is a lot of money. That is why to avoid high $RMSE$ and high R^2, we will use RMSE as our primary performance metric because it is easy to interpret and gives a clear sense of the model's accuracy.

Given the RMSE of our model, we can also form approximate confidence intervals for predictions. Assuming the residuals are roughly normally distributed, a rough 95% confidence interval for a new predicted AMRR can be approximated as :

$$\hat{y} \pm 2 \times \text{RMSE}$$

This means that for a new company, we can expect the true AMRR to fall within approximately two RMSEs of the predicted AMRR about 95% of the time. Although

this is only an approximation, it can provides a useful sense of the uncertainty associated with our model's predictions.

6 Overfitting & Underfitting

We introduced how we can measure the performance of the model. However, can we actually trust those metrics alone? The answer is no. There is a risk of overfitting and underfitting, which error metrics like RMSE or R^2 alone cannot identify. In this section, we will discuss what overfitting and underfitting mean. Then, in the next section, we will talk about how to validate our model.

Underfitting occurs when the candidate set $\mathcal{H}$ is too small or too simple to capture the general pattern in the data. For example, if our candidate set only includes linear functions but the true underlying relationship resembles a sine wave, then the model cannot generalize and will fail to capture the real pattern in the data. This leads to high misspecification error. To reduce this error, we can expand our candidate set and add more complexity.

However, if we add too much complexity, the model becomes overly flexible, leading to **overfitting**. Overfitting occurs when $\mathcal{H}$ is too large or too flexible, allowing the model to fit the noise and random variation in the training data rather than the true signal. As an example, if a model is so complex that it perfectly captures every training data point, the training RMSE will be near zero. However, when the model is tested on new, unseen data, it cannot generalize and the out-of-sample error will be high. This results in high estimation error.

In our context, currently, since we define $\mathcal{H}$ as only linear relationships, the misspecification error will be large and underfitting is a concern. Overfitting is not currently an issue because a simple linear model cannot easily memorize noise. In our modeling approach, we aim to balance these two risks. We will expand $\mathcal{H}$ to include polynomial transformations, logarithmic transformations, and interaction terms, but we will carefully control complexity to avoid overfitting. We will discuss this strategy further in the model selection section.

7 Model Validation

In this section, we will discuss how to detect overfitting and validate the model. Model validation is the process used to obtain honest estimates of a model's predictive error. As discussed earlier, overfitting occurs when the candidate set $\mathcal{H}$ is too expansive and the model fits the training data too closely, including noise. Although in-sample error metrics like RMSE may appear very low in such cases, the model may perform poorly on unseen data.

The challenge is that future data are unknown at training time. To simulate the presence of future data, we split our historical data $\mathbb{D}$ into a training set $\mathbb{D}_{\text{train}}$ and a testing set $\mathbb{D}_{\text{test}}$. We use $\mathbb{D}_{\text{train}}$ to train the model and compute in-sample RMSE, and $\mathbb{D}_{\text{test}}$ to evaluate the model and compute out-of-sample RMSE.

While the in-sample RMSE will almost always decrease as more features are added (even irrelevant ones), the out-of-sample RMSE may initially decrease and then start increasing again if unnecessary features are included. Thus, if the out-of-sample RMSE is higher than the in-sample RMSE, this is a strong indication that the model is overfitting.

To formally define the data split, we introduce a ratio K such that:

$$K = \frac{n_{\text{test}}}{n}$$

where n_{test} is the number of rows in the testing set and n is the total number of observations. For example, if $K = 5$, then 20% of the data will be reserved for testing and 80% will be used for training.

Another important point to be aware of is that if we choose a specific 20% of the data once, the out-of-sample RMSE (oosRMSE) may be biased and unstable. The estimate could vary greatly depending on which particular rows were selected for the testing set. To make the error estimate more stable and reliable, we use **cross-validation**.

In cross-validation, instead of using a single fixed testing set, we repeatedly split

the data into training and testing sets K times, each time using a different subset of the data for testing. This yields a collection of prediction errors across all folds, from which we can compute a stable estimate of the oosRMSE. In our modeling, we will use both training/testing splits and cross-validation techniques to obtain honest and reliable assessments of model performance.

When predicting for new future data, it is important to distinguish between interpolation and extrapolation. **Interpolation** occurs when the feature values of the new data fall within the range of values observed in the training set. **Extrapolation** occurs when the feature values lie outside the range of the training data. For example, if a feature value in the training data ranges from 1 to 10 and we attempt to predict using a feature value of 20, we are extrapolating.

Extrapolation is riskier because the model's predictions could be significantly inaccurate outside the regions of the training data. Therefore, extrapolation should be avoided whenever possible, and predictions should ideally be made through interpolation.

When our model is used in the real world, we expect that most users will be interpolating, as new companies' feature values are likely to fall within the range observed in our training data. Since we have trained the model on a diverse set of SaaS companies, including both large established firms and small startups, our feature space should cover most common scenarios. However, as companies evolve and new businesses emerge, it is possible that some feature values, such as the number of active users, could exceed the ranges seen during training, leading to potential extrapolation.

8 Model Selection

As discussed in Section 6, if we choose a model that is too simple, we risk underfitting and missing important patterns in the data. If we choose a model that is too complex and expansive, we risk overfitting and capturing noise. In Section 7, we discussed how to detect overfitting. Now, in this section, we turn to the problem of balancing these two risks, a task known as **model selection**. Model selection refers to choosing the best

model from a large set of possible models, balancing the need for flexibility to capture true patterns without overfitting to noise. Since many models can fit the training data well, the goal is to select the model that generalizes best to unseen data.

In our modeling approach, we plan to use **forward stepwise selection** to choose the best model from an expanded candidate set. First, we need an expansive candidate set because the true relationship between AMRR and our features is unlikely to be perfectly linear. To capture potential non-linearities, we will expand our hypothesis set to include polynomial transformations (such as x^2, x^3), logarithmic transformations (such as $\log(x)$), trigonometric transformations (such as $\sin(x)$), and interaction terms (such as $x_1 \times x_2$, $x_1 \times x_2 \times x_3$).

We then split the data $\mathbb{D}$ into three parts: a training set $\mathbb{D}_{\text{train}}$, a selection set $\mathbb{D}_{\text{select}}$, and a testing set $\mathbb{D}_{\text{test}}$. We previously introduced the roles of training and testing sets; the selection set is used to estimate out-of-sample error (oosRMSE) during the model selection process.

In forward stepwise selection, we begin by fitting an OLS model using one feature from the candidate set and training on $\mathbb{D}_{\text{train}}$. We predict on $\mathbb{D}_{\text{select}}$ and compute the resulting oosRMSE. This procedure is repeated for each feature individually. We select the feature that achieves the lowest oosRMSE and permanently include it in the model. Then, holding this feature fixed, we repeat the process: fitting models with one additional candidate feature at a time and choosing the next feature that most reduces oosRMSE.

This process continues for F steps (where F is a pre-determined maximum), or we can stop earlier if the oosRMSE on the selection set begins to increase. If oosRMSE increases, it signals that we are overfitting the model and should stop adding features.

After completing forward stepwise selection, we refit an OLS model using the selected features on the combined $\mathbb{D}_{\text{train}} \cup \mathbb{D}_{\text{select}}$ dataset. We then predict on the independent testing set $\mathbb{D}_{\text{test}}$ to obtain an honest estimate of future predictive performance. Finally, we retrain the model on the full dataset (train + select + test) using the selected features to form our final predictive model, denoted g_{final}.

g_{final} is ultimately selected because the stepwise procedure systematically tests different

models, adds complexity only when it improves predictive performance, and stops before overfitting occurs. Thus, g_{final} is expected to generalize well to new SaaS companies.

Given our dataset size of approximately 5,000 to 10,000 companies, we currently do not have a sufficient number of samples as we are going to use an expensive set of hypothesis functions which can exceed or near to 5,000 as it could be overfit to the model. However, for now, we assume that we have enough data and our transformed features from the model selection are significantly less than our number of data.

Although in principle we could have used other algorithms, such as Random Forests, we focus on ordinary least squares (OLS) because of its simplicity and interpretability. While Random Forests may offer higher predictive accuracy, OLS allows us to explain how each feature contributes to AMRR, which is crucial for understanding the drivers of company performance.

9 Making Predictions

Now, we have our final model g_{final} to make prediction for new observations. Prediction using a model means that we apply a new input vector $\boldsymbol{x}^*$, which contains new measurements for same features, to a function g_{final}. For example, suppose we are given a new company with the following feature values:

$$x^* = \begin{bmatrix} 12.5 & 9800 & 135.20 & 42.0 & 18.7 \end{bmatrix}$$

This vector corresponds to a churn rate of 12.5%, 9,800 active users, a CAC of \$135.20, 42% of users on premium plans, and 18.7% revenue growth. We compute the predicted AMRR by evaluating the function:

$$\hat{y} = g_{\text{final}}(x^*)$$

In ordinary least squares (OLS), this corresponds to taking a weighted sum of the input features, where each coefficient reflects the marginal effect of that feature on AMRR.

Unlike more complex models such as Random Forests, OLS is fully interpretable: we can understand how each feature contributes to the prediction.

However, users of the model, such as startup founders or investors, may not have technical expertise. To build trust and support decision-making, we can report useful summaries such as the model's average prediction error (RMSE), confidence intervals and interpolation range. These tools help non-technical users interpret the model's predictions, even if they do not understand the internal mechanics.

10 Conclusions

To recap the essay, we developed a predictive model for estimating a SaaS company's average monthly recurring revenue (AMRR) based on a small set of core business feature metrics: customer churn rate, number of active users, customer acquisition cost, percentage of users on premium plans, and revenue growth rate. Since both our features and response are numeric and for easy interpretability, we used ordinary least squares along with an expansive hypothesis set that included feature transformations and interactions. We applied forward stepwise selection for model selection to balance underfitting and overfitting, and validated our model using out-of-sample performance to ensure that the final model generalizes well to new data.

With this limited feature set, our model cannot capture every factor that influences AMRR. However, it can offer a general sense of monthly revenue. Business stakeholders using the model should be aware of its reliability by analyzing confidence intervals, out-of-sample RMSE, and whether new inputs fall within the interpolation range.

To conclude with the performance of the model: estimation error will be moderate, since the number of rows is limited (about 5,000 to 10,000). Misspecification error is expected to be low, due to the use of forward stepwise selection. Ignorance error remains high, as we only use five features and there must be many other features that contribute to our response AMRR. The title of this essay, *The Average Monthly Recurring Revenue of Software as a Service Companies Remains Difficult to Predict*, reflects the ongoing

challenge of predicting AMRR using a small set of features and limited data.